

Collaborative assignment Report

BY: ADITYA

UPADHYAY

Intro

I used as mentioned MOVIE_LENS 100K dataset and to follow guidelines utilised the techniques of neighbourhood based .

Baselines i used First Random Eye closing and then userbased , item based and combined , i also tried content based (CBF) based models.

Section 1

```

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from typing import List, Tuple
from collections import defaultdict

np.random.seed(42)
torch.manual_seed(42)
#loader fn
# Utility functions
Tabnine | Edit | Test | Explain | Document
def rat_loader(path):
    fl = []
    with open(path, 'r') as f:
        for line in f:
            parts = line.strip().split('\t')
            if len(parts) >= 3:
                fl.append((int(parts[0]), int(parts[1]), float(parts[2])))
    return fl

Tabnine | Edit | Test | Explain | Document
def matrix_maker(base_path, test_path):
    tr_dt = rat_loader(base_path)
    tst_dt = rat_loader(test_path)
    full_dt = tr_dt + tst_dt
    max_u = max(u for u, i, r in full_dt)
    max_i = max(i for u, i, r in full_dt)
    fl_mat = np.zeros((max_u, max_i))
    for u, i, r in tr_dt:
        fl_mat[u-1, i-1] = r
    test_list = [(u-1, i-1, r) for u, i, r in tst_dt]
    return fl_mat, test_list, max_u, max_i

Tabnine | Edit | Test | Explain | Document
def compute_baseline(R):
    mask = R > 0
    mu = R[mask].mean() if mask.sum() > 0 else 3.0
    n_u, n_i = R.shape
    bu = np.zeros(n_u)

```

This snippet is essentially working as utility function for rest of the code ,

Data Loading, Baseline Modeling, and Evaluation Metrics

This section describes the common utilities used across all models in our experiments, including data loading, baseline

bias computation, and evaluation metrics. These components ensure that all recommendation models are trained and evaluated in a consistent and fair manner.

Before applying any neighborhood-based prediction, we compute a baseline rating model that captures global and individual biases. This baseline consists of:

- A global average rating μ
- User-specific biases b_u
- Item-specific biases b_i

The intuition behind this model is simple:

Some users consistently rate higher or lower than average, and some items are generally liked or disliked. Ignoring these effects can lead to biased similarity estimates and poorer predictions.

$$\hat{r}_{ui}^{\text{base}} = \mu + b_u + b_i$$

(Credits ChatGPT for image)

Evaluation Metrics

To evaluate recommendation performance, we use three standard error metrics:

- **Mean Absolute Error (MAE)**
Measures the average absolute difference between predicted and true ratings.
- **Root Mean Squared Error (RMSE)**
Penalizes larger errors more strongly and highlights outlier mistakes.
- **Normalized Mean Absolute Error (NMAE)**
Since MovieLens ratings lie in the range [1,5], MAE is normalized by the maximum possible range (4) to obtain a scale-independent metric

$$\text{NMAE} = \text{MAE}/4$$

Section 2

User based CF

This section describes the implementation of the user-based neighborhood collaborative filtering model used in our experiments. The model predicts unknown ratings by leveraging similarities between users based on their past rating behavior.

Model Overview

The user-based collaborative filtering approach assumes that users with similar rating patterns will rate unseen items similarly. For a given user–item pair (u,i), the prediction is

obtained by aggregating ratings from users who are most similar to user uu and who have already rated item iii .

To improve robustness, the model operates on bias-corrected residuals rather than raw ratings and applies shrinkage to penalize similarities computed from very few co-rated items.

Bias-Centered Residuals

Before computing similarities, systematic effects are removed from the rating matrix. Each rating is decomposed into:

- Global mean rating
- User bias
- Item bias

The residual rating matrix is defined as:

$$\hat{r}_{ui} = \mu + b_u + b_i + \frac{\sum_{v \in N_k(u,i)} s_{uv}(r_{vi} - \mu - b_v - b_i)}{\sum |s_{uv}|}$$

Only observed ratings are used, while unobserved entries are set to zero. This ensures that similarity computation focuses on relative user preferences rather than absolute rating values.

User–User Cosine Similarity

After computing the residual matrix, user–user cosine similarity is calculated. Each user is represented as a vector of residual ratings across all items. The similarity between two users is computed using the standard cosine similarity formula:

$$\text{sim}(a,b) = (a.b)/(|a||b|)$$

This similarity captures how similarly two users rate items overall and forms the core of the neighborhood selection process.

Similarity Shrinkage (Got this idea [Shrinkage_IEEE](#))

Raw similarity values can be unreliable when two users have rated only a small number of common items. To address this, shrinkage is applied based on the number of co-rated items.

This penalizes similarities computed from limited overlap and helps in producing more stable and accurate similarity estimates, which directly improves prediction quality.

Neighbourhood Selection

For a target prediction (u,i), the neighborhood consists of users who have already rated item i. Among these candidate users, the top-k users with the highest absolute similarity values to user uu are selected. Users with similarity below a minimum threshold may be discarded.

Here, k is treated as a hyperparameter and is tuned during experimentation.

Final Prediction Formula

The final predicted rating is computed by adding the baseline estimate to a weighted sum of residual ratings from similar users. The exact prediction formula is implemented directly in the code.

If no valid neighbors are found, or if the total similarity weight is very small, the model falls back to the baseline prediction.

Confidence Measure

Along with each predicted rating, a confidence score is also computed. This confidence is defined as the sum of absolute similarity weights of the selected neighbors and indicates how reliable the prediction is.

This confidence value is later used in hybrid models to adaptively combine predictions from user-based and item-based approaches.

```
... Fold u1: NMAE=0.182450, MAE=0.729799, RMSE=0.934754
Fold u2: NMAE=0.180128, MAE=0.720512, RMSE=0.925818
Fold u3: NMAE=0.179835, MAE=0.719341, RMSE=0.921065
Fold u4: NMAE=0.180497, MAE=0.721988, RMSE=0.923843
Fold u5: NMAE=0.181163, MAE=0.724652, RMSE=0.921621

=== AVERAGE (UserBasedCF) ===
Avg NMAE: 0.180815
Avg MAE: 0.723258
Avg RMSE: 0.925420
```

```
=====
=====
```

Section 3 Item based CF

This section describes the implementation of the item-based neighborhood collaborative filtering model used in our experiments. The model predicts unknown ratings by leveraging similarities between items based on how users have rated them in the past.

Model Overview

The item-based collaborative filtering approach assumes that items which receive similar ratings from users will be rated similarly by the same user. For a given user-item pair (u,i) ,

the prediction is obtained by aggregating the ratings given by the same user u to items that are most similar to item i .

Similar to the user-based approach, to improve robustness, the model operates on bias-corrected residuals rather than raw ratings and applies shrinkage to penalize similarities computed from very few co-rated users.

Bias-Centered Residuals

Before computing item–item similarities, we remove systematic effects from the rating matrix. Each rating is decomposed into:

- Global mean rating
- User bias
- Item bias

The residual rating matrix is defined in the same way as in user-based CF, where unobserved ratings are set to zero.

Item–Item Similarity

After computing the residual matrix, we transpose it to obtain item rating vectors. Item–item similarity is then computed

using the cosine similarity formula, which is very important in this model.

The cosine similarity between two items aaa and bbb is defined as:

$$\text{sim}(a,b) = (a.b)/(|a||b|)$$

This measures how similar two items are based on the pattern of user ratings they receive.

Similarity Shrinkage (Got this idea [Shrinkage IEEE](#))

Raw item–item similarity values can be unreliable when two items have been rated by only a few common users. To address this, we apply shrinkage based on the number of co-rating users. This helps in reducing noise and gives more reliable similarity estimates, which improves overall prediction accuracy.

Neighbourhood Selection

For a target prediction (u,i), the neighborhood consists of items already rated by the user . Among these candidate items, we select the top-k items with the largest absolute similarity values with respect to item i. We also optionally discard items whose similarity is below a minimum threshold. Here, k is treated as a hyperparameter.

Final Prediction Formula

The final rating prediction is obtained by adding a weighted combination of residuals from similar items to the baseline estimate. The exact formula used is the same as implemented in the code.

Confidence Measure

Along with each prediction, we also compute a confidence measure, defined as the sum of absolute similarity weights of the selected neighbors. This value indicates how confident the model is about a particular prediction and is later used in hybrid models to combine multiple predictors.

Results

```
Fold u1: NMAE=0.181707, MAE=0.726830, RMSE=0.930370
Fold u2: NMAE=0.178981, MAE=0.715923, RMSE=0.918691
Fold u3: NMAE=0.178928, MAE=0.715714, RMSE=0.916686
Fold u4: NMAE=0.179080, MAE=0.716322, RMSE=0.917175
Fold u5: NMAE=0.180248, MAE=0.720992, RMSE=0.917995
```

```
=== AVERAGE (ItemBasedCF) ===
Avg NMAE: 0.179789
Avg MAE: 0.719156
Avg RMSE: 0.920183
```

```
=====
=
```

Section 4

Weighted Hybrid Collaborative Filtering

This section describes the implementation of the weighted hybrid collaborative filtering model, which combines user-based and item-based neighborhood approaches in order to improve prediction accuracy.

Instead of relying on a single neighborhood model, this approach leverages the strengths of both user-based and item-based collaborative filtering by combining their predictions using a weighted scheme.

Model Overview

The weighted hybrid model assumes that user-based and item-based models provide complementary information. For a given user–item pair (u,i) , predictions are obtained independently from the user-based and item-based collaborative filtering models and then combined using a weighted average.

This helps in handling cases where one model performs better than the other, such as sparse user histories or items with limited ratings.

Fixed Weight Mode

In the fixed mode, the value of alpha is kept constant for all predictions. This weight is treated as a hyperparameter and determines how much importance is given to user-based versus item-based predictions across the entire dataset.

This mode is simple and stable, but it does not adapt to the reliability of individual predictions.

Confidence-Based Adaptive Mode

In the confidence-based mode, the weight is computed dynamically for each prediction. Both the user-based and item-based models return a confidence value, defined as the sum of absolute similarity weights used during prediction.

The adaptive weight is computed as:

Linear combinations of user and item based

If both confidence values are very small, the model defaults to equal weighting. This adaptive mechanism allows the hybrid model to rely more on the component that has stronger neighborhood support for a given prediction.

Hyperparameter Optimization using Optuna

The performance of neighborhood-based models is highly sensitive to hyperparameter choices. To systematically select optimal values, Optuna is used for hyperparameter tuning.

The following hyperparameters are tuned using Optuna:

- **Number of neighbors k**
- **Shrinkage parameter**
- **Minimum similarity threshold**
- **Hybrid weight**
- **Hybrid mode**

For each Optuna trial, the model is evaluated using 5-fold cross-validation on the MovieLens 100K dataset, and NMAE is used as the optimization objective. Median pruning is applied to terminate poorly performing trials early, improving search efficiency.

Final Evaluation

After selecting the best hyperparameters from Optuna, the hybrid model is retrained and evaluated across all folds. The final performance is reported using NMAE, MAE, and RMSE, along with per-fold results to ensure consistency.

=====

===

Results

```
=== Best hyperparameters (Optuna) ===
K: 140
shrink: 55
min_sim: 0.0008066302723931358
alpha: 0.49187687068808117
mode: conf

=== Final performance with best hyperparameters ===
Average NMAE: 0.179290
Average MAE: 0.717161
Average RMSE: 0.917595

Per-fold metrics (NMAE, MAE, RMSE):
Fold u1: 0.180896, 0.723586, 0.927349
Fold u2: 0.178507, 0.714030, 0.916388
Fold u3: 0.178602, 0.714409, 0.914028
Fold u4: 0.178560, 0.714241, 0.914718
Fold u5: 0.179885, 0.719538, 0.915491
```

[Generate](#)[+ Code](#)[+ Markdown](#)

=====

=====

Adaptive Hybrid Collaborative Filtering

This section describes the implementation of the adaptive hybrid collaborative filtering model, which dynamically combines user-based and item-based neighborhood approaches based on the availability and reliability of user information.

Unlike the fixed weighted hybrid model, this approach adapts its behavior per user and per prediction, making it more suitable for handling user sparsity and cold-start-like situations.

Model Overview

The adaptive hybrid model assumes that user-based and item-based collaborative filtering are not equally reliable for all users. For users with very few ratings, item-based predictions are generally more stable, while for users with rich interaction histories, user-based predictions can capture personalized preferences more effectively.

For a given user–item pair (u,i) , the model first computes predictions independently from the user-based and item-based models and then combines them using an adaptive strategy.

Adaptive Modes

The model supports three adaptive modes:

Hard Switching Mode

In the hard mode, the model switches entirely between item-based and user-based predictions based on the number of ratings provided by the user.

If the user has rated fewer items than a predefined threshold, the item-based prediction is used; otherwise, the user-based prediction is selected.

This mode provides a simple and intuitive way to handle sparse user profiles.

Soft Switching Mode

In the soft mode, the model performs a smooth interpolation between user-based and item-based predictions.

The weight assigned to the user-based model increases gradually with the number of ratings provided by the user.

This avoids abrupt switching and allows both models to contribute proportionally depending on user activity.

Confidence-Based Mode

In the confidence-based mode, the combination weight is computed dynamically using confidence values returned by the neighborhood models. These confidence values are derived from the total similarity weights used during prediction.

If both confidence values are very small, the model falls back to a simple average of the two predictions.

Prediction Strategy

The final predicted rating is computed by combining the user-based and item-based predictions using the selected adaptive mode. Predictions are clipped to the valid rating range [1,5].

This adaptive formulation allows the model to automatically emphasize the more reliable neighborhood signal for each prediction.

Batch Prediction Support

In addition to single predictions, the model also supports batch prediction for multiple items belonging to the same user. This avoids repeated computation of user statistics and improves efficiency during evaluation.

Hyperparameter Optimization using Optuna

The adaptive hybrid model contains several hyperparameters that significantly affect performance, such as:

- **Number of neighbors**
- **Shrinkage parameter**
- **Minimum similarity threshold**
- **User rating threshold**
- **Adaptive mode**

To systematically tune these parameters, Optuna is used for hyperparameter optimization.

Results

```
=== BEST HYPERPARAMETERS (Adaptive Hybrid) ===  
mode: conf  
rating_threshold: 40  
K: 30  
shrink: 75  
min_sim: 0.0
```

```
=== FINAL PERFORMANCE ===  
Avg NMAE: 0.179750  
Avg MAE: 0.719000  
Avg RMSE: 0.919939
```

```
Per-fold metrics:  
Fold u1: 0.181552, 0.726207, 0.929633  
Fold u2: 0.178914, 0.715658, 0.918972  
Fold u3: 0.178967, 0.715869, 0.916611  
Fold u4: 0.179308, 0.717230, 0.918041  
Fold u5: 0.180008, 0.720033, 0.916439
```

```
=====
```

```
=====
```

Section 6

Adaptive Collaborative Filtering with Content-Based Filtering (Adaptive CF + CBF)

I tried this specifically different technique

Motivation

Pure collaborative filtering methods rely entirely on user–item interaction data and often struggle when rating

data is sparse. On the other hand, content-based filtering uses item attributes and can provide reasonable predictions even when limited interaction data is available.

To benefit from both approaches, we combine:

- **An adaptive hybrid CF model (user-based + item-based)**
- **A content-based model using movie genre information**

The final prediction is obtained by blending the outputs of these two models.

Content-Based Filtering Model

The content-based model represents each item using its genre feature vector, extracted from the MovieLens 100K dataset. Cosine similarity is used to measure similarity between items based on these feature vectors.

For a given user–item pair (u,i) , the model predicts the rating by aggregating the user’s past ratings on similar items, using a residual-based formulation consistent with neighborhood methods.

Bias correction is applied using:

- **Global mean**
- **User bias**
- **Item bias**

This ensures that the content-based predictions remain comparable to collaborative filtering predictions.

Formula (ChatGPT for formula writing)

Let:

- $\hat{r}_{CF}(u, i)$ be the prediction from the adaptive CF model
- $\hat{r}_{CBF}(u, i)$ be the prediction from the content-based model

The final prediction is computed as:

$$\hat{r}(u, i) = (1 - w_{cbf}) \cdot \hat{r}_{CF}(u, i) + w_{cbf} \cdot \hat{r}_{CBF}(u, i)$$

where w_{cbf} controls the contribution of the content-based model.

This formulation allows the system to:

- **Rely more on collaborative filtering when interaction data is strong**
- **Fall back on content-based signals when collaborative evidence is weak**

Hyperparameter Optimization using Optuna

The combined model introduces several hyperparameters from both components, including:

- **Neighborhood size**
- **Shrinkage parameter**
- **Minimum similarity threshold**
- **User rating threshold for adaptive CF**
- **Adaptive mode (hard, soft, confidence-based)**
- **Content-based neighborhood size**
- **Content-based blending weight**

To systematically tune these parameters, Optuna is used for hyperparameter optimization.

Results

```
=== BEST CONFIG (Adaptive CF + CBF) ===
mode: conf
rating_threshold: 70
K: 30
shrink: 50
min_sim: 0.0
cbf_k: 10
w_cbf: 0.22053604727475837

=== Adaptive CF only ===
Fold u1: NMAE=0.181552, MAE=0.726207, RMSE=0.929633
Fold u2: NMAE=0.178914, MAE=0.715658, RMSE=0.918972
Fold u3: NMAE=0.178967, MAE=0.715869, RMSE=0.916611
Fold u4: NMAE=0.179308, MAE=0.717230, RMSE=0.918041
Fold u5: NMAE=0.180008, MAE=0.720033, RMSE=0.916439
AVG: NMAE=0.179750, MAE=0.719000, RMSE=0.919939

=== CBF only ===
Fold u1: NMAE=0.188217, MAE=0.752870, RMSE=0.964587
Fold u2: NMAE=0.187043, MAE=0.748171, RMSE=0.959936
Fold u3: NMAE=0.185922, MAE=0.743688, RMSE=0.951239
Fold u4: NMAE=0.186302, MAE=0.745209, RMSE=0.952237
Fold u5: NMAE=0.186730, MAE=0.746922, RMSE=0.950275
AVG: NMAE=0.186843, MAE=0.747372, RMSE=0.955655

=== Adaptive CF + CBF (Hybrid) ===
Fold u1: NMAE=0.180918, MAE=0.723673, RMSE=0.924603
Fold u2: NMAE=0.178379, MAE=0.713515, RMSE=0.915052
Fold u3: NMAE=0.178299, MAE=0.713197, RMSE=0.911535
Fold u4: NMAE=0.178664, MAE=0.714656, RMSE=0.913407
Fold u5: NMAE=0.179425, MAE=0.717701, RMSE=0.911882
AVG: NMAE=0.179137, MAE=0.716549, RMSE=0.915296
```

Final Result Table

Model	Avg NMAE	Avg MAE	Avg RMSE
User-Based CF	0.1808	0.7233	0.9254
Item-Based CF	0.1798	0.7192	0.9202
Adaptive CF (conf mode)	0.1792	0.7171	0.9175
Content-Based CF (CBF)	0.1797	0.7190	0.9199
Adaptive CF + CBF (Hybrid)	0.1791	0.7165	0.9153

Conclusion

The best results i got is on Content based adaptive hybrid model , the reason it got predictions from other sites and also optuna provided a good search space giving me good hyperparams to get a better search space to work upon .

References

- 1) [WEIGHTED HYBRID MODEL FOR IMPROVING PREDICTIVE PERFORMANCE OF RECOMMENDATION SYSTEMS USING ENSEMBLE LEARNING](#)
- 2) [Shrinkage_ IEEE](#)
- 3) [Hybrid Recommendation System Using User-based and Item-based Collaborative Filtering](#)
- 4) [Estimating Optimal Weights in Hybrid Recommender Systems](#)
- 5) [optuna.samplers](#)

